

oggetto Relazione progetto Programmazione a Oggetti
gruppo Pietro Leonardo Acampora, matricola 2146762
titolo Agenda Qt
Applicazione desktop per la gestione di attività personali

Introduzione

Agenda Qt è un'applicazione desktop sviluppata in C++17 con Qt Widgets per la gestione di attività personali. Il programma permette di creare, modificare, eliminare e organizzare attività di natura diversa, mantenendo l'obiettivo principale del progetto di Programmazione a Oggetti: costruire un modello logico non banale, separato dall'interfaccia grafica e basato sull'uso del polimorfismo.

L'utente può lavorare con quattro tipologie concrete di attività: eventi, scadenze, promemoria e checklist. Ogni tipologia condivide alcuni dati comuni, come titolo, descrizione, categoria, priorità e stato di completamento, ma possiede anche informazioni specifiche. Per esempio un evento ha un intervallo temporale e un luogo, una scadenza ha una data limite, un promemoria ha una data di notifica e una checklist contiene un insieme di elementi completabili singolarmente.

Il progetto non è pensato soltanto come una lista di attività. Sono state aggiunte funzioni pratiche come ricerca, filtri, ordinamento, categorie personalizzate, ricorrenze, template, salvataggio in JSON e undo/redo. Durante lo sviluppo ho cercato di mantenere l'interfaccia semplice, evitando una schermata troppo decorativa e dando più spazio alla lista delle attività, ai dettagli e ai controlli realmente utili.

Descrizione del modello

Il modello logico è costruito attorno alla classe astratta `Activity`, che rappresenta il concetto generale di attività. Questa classe contiene i dati comuni e dichiara alcune operazioni virtuali utilizzate dalle sottoclassi per fornire comportamenti diversi a seconda del tipo concreto. La scelta è stata quella di tenere il modello indipendente dai widget grafici: le classi del dominio non dipendono da Qt Widgets e possono quindi essere ragionate separatamente dall'interfaccia.

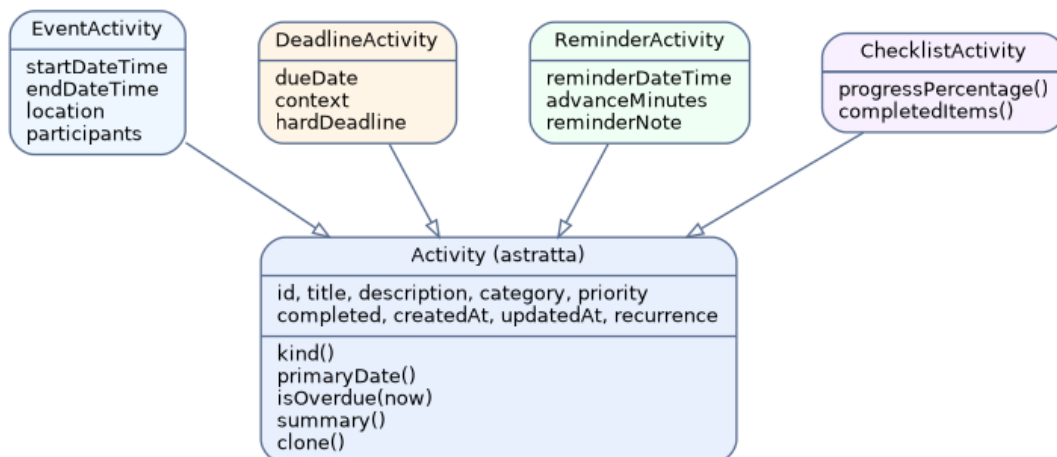


Figura 1: gerarchia principale delle attività.

La gerarchia comprende quattro classi concrete: `EventActivity`, `DeadlineActivity`, `ReminderActivity` e `ChecklistActivity`. `EventActivity` rappresenta un evento con inizio, fine, luogo e partecipanti; `DeadlineActivity`

rappresenta una scadenza con data limite e contesto; ReminderActivity rappresenta un promemoria con data e anticipo; ChecklistActivity rappresenta una lista di elementi, ciascuno con il proprio stato di completamento.

Accanto alla gerarchia principale sono presenti alcune classi di gestione. ActivityManager mantiene la collezione di attività e ne gestisce aggiunta, rimozione, aggiornamento e ricerca tramite identificatore. CategoryManager gestisce le categorie personalizzate, evitando nomi vuoti o duplicati e propagando la rinomina alle attività esistenti. ActivityTemplateManager permette di salvare attività prototipo e riutilizzarle per crearne di nuove. SearchEngine e ActivityFilter si occupano rispettivamente della ricerca e dell'applicazione dei filtri/ordinamenti.

Componente	Tipo	Ruolo nel modello
Activity	astratta	raccoglie i dati comuni e definisce le operazioni virtuali
EventActivity	concreta	gestisce eventi con intervallo temporale e luogo
DeadlineActivity	concreta	gestisce attività con una data limite
ReminderActivity	concreta	gestisce promemoria con data/ora e anticipo
ChecklistActivity	concreta	gestisce una lista di elementi completabili
ActivityManager	servizio di modello	possiede le attività tramite smart pointer e coordina le operazioni principali
SearchEngine / ActivityFilter	servizi di modello	ricerca, suggerimenti, filtri e ordinamenti

La proprietà degli oggetti Activity è gestita tramite `std::unique_ptr`. Questa scelta rende esplicito il fatto che ActivityManager possiede le attività, evitando cancellazioni manuali e riducendo il rischio di puntatori pendenti. Quando serve duplicare un'attività, per esempio per i template o per alcune operazioni di undo/redo, viene usata una clonazione polimorfica invece di copiare l'oggetto conoscendone il tipo concreto.

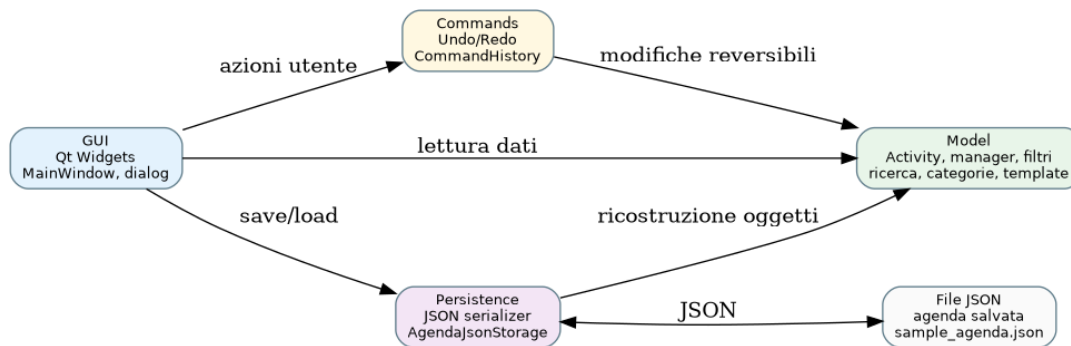


Figura 2: separazione tra GUI, comandi, modello e persistenza.

La struttura del progetto riflette questa separazione: la cartella `src/model` contiene il dominio applicativo, `src/gui` contiene le finestre e i dialog Qt, `src/persistence` contiene la conversione JSON, mentre `src/commands` raccoglie le classi usate dal sistema di undo/redo. MainWindow coordina i segnali e gli slot principali, ma non dovrebbe diventare il luogo in cui si concentra tutta la logica applicativa.

Polimorfismo

L'uso del polimorfismo nel progetto non si limita alla presenza di un distruttore virtuale nella classe base. La gerarchia Activity viene usata in modo polimorfico in diverse parti del programma: la lista delle attività contiene puntatori alla classe base, mentre il comportamento effettivo dipende dal tipo dinamico dell'oggetto.

Un primo esempio è il calcolo della data principale tramite `primaryDate()`. Ogni attività possiede un'idea diversa di data rilevante: per una scadenza è la data limite, per un promemoria è la data del promemoria, per un evento

può essere l'inizio o la fine dell'intervallo, mentre per una checklist la data è legata alla sua organizzazione interna. Filtri e ordinamenti possono quindi lavorare su Activity senza conoscere i dettagli delle sottoclassi.

Un secondo esempio è `isOverdue(now)`, che stabilisce se un'attività è scaduta confrontandola con la data corrente. Anche in questo caso la logica concreta dipende dal tipo dell'attività. Lo stato di scadenza non è scritto fisso nel programma: la data corrente viene letta dal sistema tramite Qt e confrontata con i dati dell'attività.

La funzione `clone()` è un altro punto importante. Template, aggiornamenti e comandi di undo/redo richiedono spesso di copiare un'attività senza perdere il tipo dinamico. Delegando la copia alla sottoclasse concreta si evita di usare una serie di controlli manuali sul tipo e si mantiene il codice più estensibile.

Operazione virtuale	Uso nel programma
<code>primaryDate()</code>	permette a filtri e ordinamenti di trattare attività diverse in modo uniforme
<code>isOverdue(now)</code>	calcola lo stato di scadenza con regole specifiche per tipo
<code>summary()</code>	produce una descrizione sintetica adatta alla sottoclasse concreta
<code>clone()</code>	crea copie polimorfiche senza conoscere il tipo dinamico
<code>cloneWithNewId()</code>	riusa la clonazione per creare nuove attività da template

Nel progetto è presente anche un identificatore del tipo di attività, utile soprattutto per serializzazione JSON e visualizzazione. Non è però usato come sostituto del polimorfismo per implementare i comportamenti principali: le operazioni più rilevanti rimangono virtuali e sono risolte in base al tipo dinamico.

Persistenza dei dati

La persistenza è realizzata in formato JSON. È stato scelto JSON perché è leggibile, facile da controllare durante il debug e ben supportato da Qt tramite `QJsonObject`, `QJsonArray` e `QJsonDocument`. Il programma non utilizza percorsi hardcoded: l'utente può scegliere il file da salvare o caricare tramite file dialog.

La classe `AgendaJsonStorage` coordina il salvataggio e il caricamento dell'agenda completa, mentre `ActivityJsonSerializer` si occupa della conversione delle attività polimorfiche. Nel file viene salvato un campo `type` per distinguere le varie sottoclassi, oltre ai campi comuni e ai campi specifici di ciascun tipo.

```
{
  "version": 1,
  "categories": [ ... ],
  "activities": [ ... ],
  "templates": [ ... ]
}
```

Oltre alle attività, vengono salvate anche le categorie personalizzate e i template. Questa scelta è importante perché l'utente si aspetta che un'agenda caricata mantenga non solo le attività, ma anche l'organizzazione costruita attorno a esse. Nel progetto è incluso il file `examples/sample_agenda.json`, utile sia per i test sia per la dimostrazione: contiene categorie, 16 attività, tutti i tipi concreti, ricorrenze, checklist, attività completate, attività scadute e template.

Sezione JSON	Contenuto
<code>version</code>	versione dello schema di salvataggio
<code>categories</code>	categorie personalizzate definite dall'utente
<code>activities</code>	lista di attività con campi comuni e dati specifici
<code>templates</code>	attività prototipo riutilizzabili

Funzionalità implementate

Le funzionalità implementate possono essere divise in funzionalità logiche, legate al comportamento del programma, e funzionalità grafiche, legate all'usabilità dell'interfaccia. Questa distinzione è utile perché alcune

funzioni, come la ricerca o il salvataggio, appartengono al modello e alla logica applicativa, mentre altre riguardano principalmente il modo in cui l'utente interagisce con l'applicazione.

Funzionalità logiche

- creazione, modifica, eliminazione e completamento delle attività;
- quattro tipi concreti di attività: eventi, scadenze, promemoria e checklist;
- ricerca case-insensitive, accent-insensitive e parziale;
- suggerimento fuzzy quando non viene trovato un risultato diretto;
- filtri avanzati per tipo, categoria, priorità, completamento, ricorrenza e stato di scadenza;
- ordinamento per data, titolo, priorità, completamento, creazione e aggiornamento;
- categorie personalizzate con rinomina propagata sulle attività esistenti;
- template di attività riutilizzabili;
- ricorrenze giornaliere, settimanali, mensili o annuali con condizione di termine;
- salvataggio e caricamento in formato JSON;
- tracciamento delle modifiche non salvate;
- undo/redo delle principali operazioni tramite Command pattern.

Funzionalità grafiche

- finestra principale con lista attività e pannello dettagli;
- dialog dedicati per creazione, modifica e gestione categorie;
- combo box editabile per selezionare o creare categorie nei form;
- lista attività con titolo, tipo, data, priorità, stato, categoria e ricorrenza;
- attività completate rese visivamente meno evidenti;
- attività scadute evidenziate con colore dedicato ma leggibile;
- attività ad alta priorità messe maggiormente in evidenza;
- pannello filtri richiudibile e pannello laterale scorrevole;
- messaggi di feedback dopo salvataggio e caricamento con nome file e conteggi;
- icona dell'applicazione integrata nelle risorse Qt;
- conferma prima di perdere modifiche non salvate.

Per non appesantire l'interfaccia, durante la rifinitura finale è stato rimosso un header grafico troppo grande. L'icona è rimasta come identità dell'applicazione, ma lo spazio principale è stato restituito alla lista delle attività e al pannello dei dettagli.

Scelte progettuali e pattern

La struttura generale segue una separazione simile a Model-View: il modello contiene le regole e i dati dell'applicazione, mentre la GUI raccoglie input e mostra risultati. Non ho usato un framework MVC rigido, ma ho cercato di evitare che il codice grafico diventasse responsabile di tutto.

Per undo e redo è stato introdotto il Command pattern. Ogni operazione reversibile implementa un'interfaccia comune con `execute()`, `undo()` e `description()`. `CommandHistory` mantiene due stack, uno per le operazioni annullabili e uno per quelle ripristinabili. Questa soluzione è più ordinata rispetto a una gestione manuale sparsa in `MainWindow`.

Comando	Operazione gestita
AddActivityCommand	aggiunta di una nuova attività
RemoveActivityCommand	rimozione con possibilità di ripristino
UpdateActivityCommand	sostituzione di una attività modificata
ToggleCompletionCommand	cambio dello stato di completamento

L'uso di smart pointer, in particolare `std::unique_ptr`, è stato scelto per rendere chiara l'ownership degli oggetti del modello. In un progetto con ereditarietà e oggetti creati dinamicamente, questa scelta riduce il rischio di errori di memoria e rende più semplice ragionare sulla vita degli oggetti.

Testing e validazione

Il progetto è stato testato manualmente tramite l'interfaccia grafica e tramite compilazione nell'ambiente Docker del corso. Il file `sample_agenda.json` ha permesso di verificare rapidamente molti casi diversi senza ricreare i dati a mano a ogni avvio.

Area testata	Controlli effettuati
Creazione attività	creazione di eventi, scadenze, promemoria e checklist
Modifica attività	modifica di campi comuni, specifici, checklist e ricorrenze
Categorie	aggiunta, rinomina, rimozione, clear e sincronizzazione con i filtri
Template	salvataggio come template e creazione di nuove attività da template
Ricerca e filtri	ricerca parziale, filtri combinati e ordinamenti
Persistenza	salvataggio, caricamento, messaggi di feedback e sample JSON
Undo/Redo	annullamento e ripristino di aggiunta, rimozione, modifica e completamento
Docker	compilazione con <code>qmake6</code> e <code>make</code> nell'ambiente Ubuntu/Qt del corso

Rendicontazione ore

Le ore riportate sono una stima realistica delle principali fasi di lavoro. Il monte ore effettivo è leggermente superiore a quello previsto soprattutto per la GUI e per l'integrazione tra funzionalità diverse. Alcune parti, come categorie, template e filtri, hanno richiesto più prove perché coinvolgevano sia il modello sia l'interfaccia.

Attività	Ore previste	Ore effettive
Analisi iniziale e progettazione	7	8
Implementazione modello logico e polimorfismo	10	11
Interfaccia grafica Qt	13	15
Persistenza JSON e file di esempio	6	6
Ricerca, filtri, categorie, ricorrenze e template	6	7
Undo/redo e rifiniture architetturali	3	4
Testing, debug e validazione Docker	3	4
README e relazione finale	2	2
Totale	50	57

Compilazione

Il progetto usa `qmake` e non `CMake`. Da una cartella di build separata è possibile compilare con:

```
mkdir -p build
cd build
qmake ../agenda_qt.pro
make
```

Su alcune distribuzioni Linux il comando può essere qmake6:

```
mkdir -p build
cd build
qmake6 ../agenda_qt.pro
make
```

Per validare il progetto nell'ambiente Docker del corso sono stati usati i seguenti comandi:

```
docker build -t unipd-oop/qt-env:2025 .
docker run --rm -it -v "$PWD":/workspace -w /workspace unipd-oop/qt-env:2025 bash

mkdir -p build_docker
cd build_docker
qmake6 ../agenda_qt.pro
make -j$(nproc)
```

L'avvio della GUI dentro Docker non è necessario per la validazione, perché il forwarding grafico dipende dal sistema host. La parte importante è che qmake6 e make terminino senza errori nell'ambiente standard del corso.

Conclusioni

Agenda Qt realizza un'applicazione desktop completa per la gestione di attività personali e dimostra l'uso dei principali concetti richiesti nel corso: ereditarietà, classi astratte, polimorfismo, separazione tra modello e interfaccia, gestione della memoria tramite smart pointer e persistenza dei dati.

Il progetto è stato rifinito anche dal punto di vista pratico: include un file JSON di esempio, istruzioni di compilazione, README aggiornato, validazione Docker e commenti essenziali nei sorgenti. Le funzionalità aggiuntive non sono state inserite solo per aumentare il numero di elementi presenti, ma per rendere l'applicazione più coerente e più vicina a un piccolo software realmente utilizzabile.